

BÀI TẬP VỀ NHÀ — Ôn tập Bài 1 → 6

Phạm vi ôn tập: Giai đoạn 1 (Nền tảng Angular) và Giai đoạn 2 (State và Signals)

Nguyên tắc quan trọng: KHÔNG động vào project Smart Recipe Box. Toàn bộ bài tập làm trên một project mới, để dự án chính giữ nguyên trạng thái sẵn sàng cho Bài 7.

Chuẩn bị: Tạo sân tập riêng

Mở Terminal và chạy:

```
cd <Địa chỉ khởi tạo project>
```

```
ng new bai-tap-tra-sua (chọn Tailwind CSS, chọn N cho SSR,SSG, chọn AGENTS.md)
```

Sau khi cài xong:

```
cd bai-tap-tra-sua && ng serve -o
```

 Chạy ở cổng **4200**. Mở trình duyệt tại **http://localhost:4200/**

PHẦN A — Lý thuyết

Trả lời ngắn gọn bằng lời của bạn. Ghi ra giấy hoặc một file text, không cần code.

1. Một component Angular gồm mấy file? Kể tên và cho biết mỗi file phụ trách việc gì.
2. Trong `@Component`, ba mục `selector`, `templateUrl`, `styleUrl` có ý nghĩa gì?
3. Phân biệt `{{ }}` và `( )`: mỗi cái đưa dữ liệu đi theo chiều nào?
4. Vì sao viết `{{ title() }}` mà không phải `{{ title }}`? Nếu quên cặp `( )` thì màn hình sẽ hiện gì?
5. `protected` và `readonly` trước một signal có tác dụng gì? Nếu bỏ đi thì code có chạy không?
6. Khi nào dùng `.set()`, khi nào dùng `.update()`? Cho một ví dụ đổi thường cho mỗi loại.
7. `computed()` khác signal thường ở điểm nào? Vì sao **không** gọi được `.set()` trên một computed?
8. Vì sao nên tách `interface` ra file `models.ts` riêng thay vì viết thẳng trong component?

PHẦN B — Tìm lỗi trong code

Mỗi đoạn dưới đây có **đúng một lỗi**. Hãy chỉ ra lỗi ở đâu, giải thích **tại sao sai**, và viết lại cho đúng.

Lỗi số 1

```
<h1>{{ tenCuaHang }}</h1>
```

Lỗi số 2

```
protected tangSoLy(): void {  
    soLy.update((n) => n + 1);  
}
```

Lỗi số 3

```
protected chonTraSuaMatcha(): void {  
    this.selectedDrink().set(MOCK_DRINKS[1]);  
}
```

Lỗi số 4

```
protected giamSoLy(): void {  
    this.soLy.set(this.soLy - 1);  
}
```

Lỗi số 5

```
protected readonly toppingDaNhan = computed(() =>  
    this.selectedDrink().toppings.map((topping) => {  
        name: topping.name,  
        quantity: topping.quantity * this.soLy()  
    })  
);
```

Lỗi số 6

```
protected readonly tongTien = computed(() => this.giaCoBan() *
this.solLy());

protected resetGia(): void {
  this.tongTien.set(0);
}
```

PHẦN C — Project chính: Quầy Trà Sữa

Bạn sẽ xây một ứng dụng tính tiền trà sữa. Cấu trúc giống Smart Recipe Box nhưng **dữ liệu và logic khác**, nên bạn phải thật sự hiểu chứ không chép lại được.

Nhiệm vụ 1 — ôn Bài 1

Dọn sạch trang chào mừng mặc định. Ứng dụng chỉ còn một tiêu đề `<h1>` duy nhất, và terminal không được có cảnh báo vàng nào.

Nhiệm vụ 2 — ôn Bài 2

Tên quán không được viết cứng trong HTML. Nó phải nằm trong một signal ở class và được đưa ra màn hình qua interpolation. Bạn đổi giá trị trong file `.ts` là tiêu đề trên web phải đổi theo.

Nhiệm vụ 3 — ôn Bài 4 (phần interface & mock data)

Tạo hai file `models.ts` và `mock-drinks.ts` trong `src/app/`.

`models.ts` cần **hai** interface, đều được export:

- `Topping` — gồm `name` (chữ), `quantity` (số), `unit` (chữ)
- `DrinkModel` — gồm `id` (số), `name` (chữ), `description` (chữ), `giaCoBan` (số), và `toppings` là một danh sách `Topping`

`mock-drinks.ts` export hằng số `MOCK_DRINKS` chứa **ba** loại trà sữa do bạn tự nghĩ ra. Gợi ý: Trà sữa trân châu đường đen, Trà sữa Matcha, Hồng trà sữa. Mỗi loại có `giaCoBan` khác nhau (khoảng 30000–55000) và 2–3 topping.

⚠ Nhớ quy ước: tên hằng số VIẾT_HOA_CÓ_GẠCH_DƯỚI, và file này phải `import` interface từ `./models`.

Nhiệm vụ 4 — ôn Bài 3 + Bài 4

Màn hình có **ba nút**, mỗi nút chọn một loại trà sữa. Ứng dụng luôn nhớ loại đang được chọn và hiển thị **tên, mô tả, giá cơ bản** của nó. Bấm nút nào thì nội dung đổi ngay sang loại đó.

Nhiệm vụ 5 — ôn Bài 5

Thêm bộ đếm **số ly** với hai nút **+** và **-**, hiển thị con số hiện tại ở giữa. Giá trị khởi đầu là 1 và **không được tụt xuống dưới 1**. Hai nút chỉ có ký hiệu nên nhớ phần dành cho người khiếm thị (**aria-label**).

Nhiệm vụ 6 — ôn Bài 6

Đây là phần quan trọng nhất. Bạn cần **hai** dữ liệu dẫn xuất, và cả hai đều phải tự động cập nhật mà bạn không viết code cập nhật trong các hàm bấm nút:

- tongTien** — một **con số**: giá cơ bản của loại đang chọn nhân với số ly. Hiển thị bằng interpolation bình thường.
- toppingCanDung** — một **danh sách**: các topping của loại đang chọn, với số lượng đã nhân theo số ly. Hiển thị tạm bằng mẹo `<pre>{{ ... | json }}</pre>`.

Cách tự chấm điểm: Bấm **+** vài lần rồi đổi loại trà sữa qua lại. Nếu cả tổng tiền lẫn danh sách topping đều luôn khớp đúng mà bạn **chưa từng viết một dòng cập nhật nào** trong 5 hàm bấm nút → bạn đã hiểu `computed()`. 

PHẦN D — Thử thách nâng cao (bắt buộc)

Làm được thì rất giỏi, không làm được thì phải làm được:

- Định dạng tiền tệ.** `55000` đọc khó. Tìm hiểu pipe `number` của Angular để hiện thành `55,000`. (Gợi ý: cú pháp giống `| json` bạn đã biết, và cũng cần `import` một thứ vào mảng `imports`.)
- Giảm giá theo số lượng.** Thêm một `computed` nữa: mua từ 5 ly trở lên thì được giảm 10% tổng tiền. (Gợi ý: bên trong `computed` bạn vẫn viết được lệnh `if`.)
- Đếm tổng số topping.** Tạo một `computed` trả về tổng số lượng của tất cả topping cộng lại. (Gợi ý: tìm hiểu phương thức `.reduce()` của mảng — đây là anh em với `.map()`.)

Bảng tự theo dõi tiến độ

Phần	Nội dung	Đã xong?
A	8 câu lý thuyết	☐
B	6 đoạn code tìm lỗi	☐
C1	Dọn trang mặc định + <code><h1></code>	☐
C2	Signal tên quán + interpolation	☐
C3	<code>models.ts</code> + <code>mock-drinks.ts</code>	☐
C4	Ba nút chọn trà sữa + <code>.set()</code>	☐
C5	Bộ đếm số ly + <code>.update()</code>	☐
C6	Hai <code>computed()</code>	☐

Phần	Nội dung	Đã xong?
D1	Định dạng tiền tệ	☐
D2	Giảm giá từ 5 ly	☐
D3	Tổng số topping với <code>.reduce()</code>	☐

Sổ tay tra cứu nhanh (Bài 1 → 6)

Ba ký hiệu cốt lõi

Ký hiệu	Tên	Chiều	Dùng khi
<code>{{ }}</code>	Interpolation	Class  Màn hình	Hiện chữ giữa hai thẻ
<code>[]</code>	Property binding	Class  Màn hình	Gán vào thuộc tính của thẻ
<code>()</code>	Event binding	Màn hình  Class	Bắt sự kiện người dùng

Signal

```
// Khai báo
protected readonly count = signal<number>(0);
protected readonly name = signal('Lan'); // TypeScript tự suy ra kiểu string

// Đọc – luôn có cặp ()
count()

// Ghi đè giá trị mới (đã biết trước giá trị)
this.count.set(10);

// Tính từ giá trị cũ
this.count.update((current) => current + 1);
```

Computed

```
import { Component, computed, signal } from '@angular/core';

protected readonly total = computed(() => this.price() * this.quantity());

// ✅ Đọc được: total()
// ❌ KHÔNG ghi được: total.set(...)
```

Method trong class

```
protected doSomething(): void {  
  console.log('Đã chạy');  
}
```

Interface & mock data

```
// models.ts  
export interface Book {  
  title: string;  
  pageCount: number;  
}  
  
// mock-books.ts  
import { Book } from './models';  
  
export const MOCK_BOOKS: Book[] = [{ title: 'Đế Mèn Phiêu Lưu Ký',  
  pageCount: 150 }];
```

.map() — biến đổi cả mảng

```
const doubled = numbers.map((n) => n * 2);  
  
// Trả về đối tượng: BẮT BUỘC bọc trong cặp ngoặc tròn ({ ... })  
const sale = products.map((p) => ({  
  name: p.name,  
  price: p.price / 2,  
}));
```

Hiển thị mảng tạm thời

```
import { JsonPipe } from '@angular/common';  
  
@Component({  
  imports: [JsonPipe],  
  // ...  
})
```

```
<pre>{{ myArray() | json }}</pre>
```

Những cái bẫy hay dính

Bẫy	Sai	Đúng
Quên <code>()</code> khi đọc signal	<code>{{ title }}</code>	<code>{{ title() }}</code>
Quên <code>this.</code> trong class	<code>count.set(1)</code>	<code>this.count.set(1)</code>
Gọi <code>.set()</code> sai chỗ	<code>count().set(1)</code>	<code>count.set(1)</code>
Arrow function trả về object	<code>=> { a: 1 }</code>	<code>=> ({ a: 1 })</code>
So sánh lỏng lẻo	<code>if (x == 1)</code>	<code>if (x === 1)</code>
Ghi vào computed	<code>total.set(0)</code>	Sửa signal nguồn
Nút chỉ có ký hiệu	<code><button>+</button></code>	thêm <code>aria-label</code>